OSSP PRACTICAL EXPERIMENTS — ALL 4 EXPERIMENTS

Operating Systems and Systems Programming (25CS2104E)

EXPERIMENT 1

Code

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[50];

    printf("Enter command: ");
    fflush(stdout);
    scanf("%49s", command);

    int pid = fork();

    if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());
        fflush(stdout);
        execlp(command, command, NULL);
        perror("exec failed");
    }
    else
    {
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        fflush(stdout);
        wait(NULL);
        printf("Child completed\n");
    }

    return 0;
}
```

Output

```
Enter command: ls
Parent PID: 434
Child PID: 435
Child PID: 435
copy.txt
exp1
exp1.c
exp2
exp2.c
exp3
exp3.c
exp4
exp4.c
out1.txt
source.txt
Child completed

Additional Linux commands:
uname -a
lscpu
lsblk
ps -f
top

These commands were used to observe OS, CPU, storage and process information.
```

Execution

```
gcc exp1.c -o exp1
./exp1
```

EXPERIMENT 2

Code

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int source, destination;
    char buffer[100];
    int n;

    source = open("source.txt", O_RDONLY);
    destination = open("copy.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    n = read(source, buffer, sizeof(buffer));

    write(destination, buffer, n);

    close(source);
    close(destination);

    printf("File copied successfully\n");

    return 0;
}
```

Output

```
File copied successfully

Contents of copy.txt:
Hello, this is my source file.

strace:
The execution environment did not have the strace utility installed, so no real strace output was generated.
```

Execution

```
gcc exp2.c -o exp2
./exp2
```

EXPERIMENT 3

Code

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pid = fork();

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("PID: %d\n", getpid());
        printf("PPID: %d\n", getppid());
        fflush(stdout);
        sleep(5);
        printf("Child completed\n");
        fflush(stdout);
    }
    else
    {
        printf("Parent Process\n");
        printf("PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        fflush(stdout);
        wait(NULL);
        printf("Parent completed\n");
    }

    return 0;
}
```

Output

```
Parent Process
PID: 438
Child PID: 439
Child Process
PID: 439
PPID: 438
Child completed
Parent completed

Observation:
The child has a different PID and its PPID matches the parent's PID.
The child sleeps for 5 seconds so its process can be observed using ps/top or /proc.
```

Execution

```
gcc exp3.c -o exp3
./exp3
```

EXPERIMENT 4

Code

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pid1, pid2;

    pid1 = fork();

    if (pid1 == 0)
    {
        printf("Child 1 PID: %d\n", getpid());
        fflush(stdout);
        sleep(2);
        printf("Child 1 completed\n");
        return 0;
    }

    pid2 = fork();

    if (pid2 == 0)
    {
        printf("Child 2 PID: %d\n", getpid());
        fflush(stdout);
        sleep(3);
        printf("Child 2 completed\n");
        return 0;
    }

    printf("Parent PID: %d\n", getpid());
    fflush(stdout);

    wait(NULL);
    printf("One child completed\n");
    fflush(stdout);

    waitpid(pid2, NULL, 0);
    printf("Second child completed\n");

    return 0;
}
```

Output

```
Parent PID: 440
Child 1 PID: 441
Child 2 PID: 442
Child 1 completed
One child completed
Child 2 completed
Second child completed

Zombie demonstration:
Child completed
Parent sleeping...

Example process state:
PID    PPID  S  COMMAND
451    449   Z  zombie
```

Execution

```
gcc exp4.c -o exp4
./exp4
```